Dictionnaire des méthodes — Projet 8 : Gestion du personnel et de la paie

Ce document explique, pour chacune des 84 méthodes du diagramme de classes UML, **à quoi elle sert** (description) et **comment elle fonctionne** (étapes). Il accompagne le diagramme `Diagramme_Classes_UML_unique_v3.drawio`.

1. Dans quel langage sont écrites les méthodes ?

En PHP, orienté objet (PHP 8), avec une base **MySQL** accessible par **PDO** (requêtes préparées). C'est le langage imposé par le projet. Chaque classe du diagramme devient une classe PHP, chaque méthode devient une méthode de cette classe. Toute la logique métier (règles, calculs, droits, accès à la base) s'exécute **sur le serveur**.

Le **JavaScript** n'écrit aucune méthode du diagramme. Il sert seulement dans le navigateur, pour les échanges dynamiques (AJAX) : recherche d'employés pendant la frappe, durée d'un congé affichée en direct, liste des jours fériés. Le navigateur appelle alors un petit script PHP, qui appelle la méthode voulue.

Trois bibliothèques PHP prennent en charge les opérations techniques :

Besoin	Bibliothèque
Envoyer un e-mail	PHPMailer
Produire un PDF (bulletin de paie)	Dompdf
Produire un fichier Word (contrat, attestation)	PhpWord

Les méthodes du diagramme **appellent** ces bibliothèques ; elles ne les réécrivent pas.

Correspondance des types UML vers PHP

Le diagramme utilise des types neutres. En PHP :

Type du diagramme	En PHP
`String`	`string`
`int`	`int`
`boolean`	`bool`
`decimal`	`float` arrondi à 2 décimales (la base stocke en `DECIMAL`)
`Date`	`DateTime` (ou texte `AAAA-MM-JJ`)
`List<Classe>`	tableau (`array`) d'objets de cette classe
`File`	le fichier produit, ou son chemin
`void`	la méthode ne retourne rien
`Role`, `StatutConge`…	valeurs fixes (énumérations) : voir le diagramme

2. Comment lire une méthode

- `+` : méthode publique, appelable depuis les autres classes et les pages. `-` : méthode privée, étape interne de la classe.
- **Statique** (soulignée dans le diagramme) : s'appelle sur la classe, sans objet, parce qu'on n'en a pas encore. Exemple : `Utilisateur::authentifier($email, $mdp)` (on se connecte justement pour obtenir l'utilisateur).
- **Normale** : s'appelle sur un objet déjà chargé. Exemple : `$demande->soumettre()`.
- Le type après les deux-points est ce que la méthode **retourne**.

3. Règles communes à toutes les méthodes

- **Qui agit.** Quand une méthode change quelque chose d'important (paie, congé, compte), elle reçoit en paramètre l'`auteur`, le `chef`, la `rh`… c'est-à-dire l'utilisateur connecté. Cette personne vient **toujours de la session**, jamais d'un champ de formulaire. Elle sert à contrôler le droit d'agir et à remplir le journal.
- **Droits.** Avant une action sensible, la méthode vérifie le droit (`Utilisateur.peut()`, ou une règle précise : le chef de l'équipe, la RH…). Si l'action est interdite, elle s'arrête avec une erreur.
- **Erreurs.** Une règle non respectée (solde insuffisant, chevauchement, droit manquant) déclenche une exception PHP. La page l'attrape et affiche un message clair.
- **Transaction.** Quand plusieurs écritures vont ensemble (décision d'un congé, génération d'une fiche), elles sont faites dans une seule transaction : tout est enregistré, ou rien.

- **Journal et notifications.** Les actions sensibles sont notées dans `JournalAction` ; les personnes concernées sont prévenues par `Notification.notifier`.
- **Jamais de suppression.** Comptes, types de congé et rubriques se désactivent ; le passé reste lisible.
- **Opérations standard non listées.** Créer, charger, enregistrer et lister un objet existent bien, mais elles sont identiques partout et ne portent aucune règle : elles ne figurent pas dans le diagramme. Seules les méthodes qui portent une règle du projet sont décrites ici.

4. Vue rapide

- 84 méthodes pour 25 classes.
- 12 méthodes statiques : `Utilisateur.inscrire`, `Utilisateur.authentifier`, `Utilisateur.demanderReinitialisation`, `JetonReinitialisation.trouverParJeton`, `Employe.rechercher`, `JourFerie.estOuvrable`, `JourFerie.getFeriesEntre`, `Absence.marquer`, `Notification.notifier`, `JournalAction.enregistrer`, `JournalAction.rechercher`, `ParametreSysteme.getDecimal`.
- 2 méthodes privées : `SoldeConge.getJoursEnAttente`, `DemandeConge.determinerStatutInitial`.

Circuit d'une demande de congé (utile pour lire `DemandeConge`)

Qui demande	Statut de départ	Avis du chef	Décision finale
Employé	`en_attente_chef`	oui (`donnerAvis`)	RH (`decider`)
Chef	`en_attente_rh`	non	RH (`decider`)
Membre de la RH	`en_attente_dg`	non	DG (`decider`)

Statuts possibles : `en_attente_chef`, `en_attente_rh`, `en_attente_dg`, `approuvee`, `refusee`, `annulee`. Un décideur n'est jamais le demandeur.

5. Les méthodes, classe par classe

Utilisateur

Le compte de connexion de toute personne (employé, chef, RH, DG). Il porte le rôle et l'état du compte (en attente, actif, désactivé).

`+ inscrire(email : String, motDePasse : String, role : Role) : Utilisateur` — ***statique***

Description. Crée un nouveau compte quand quelqu'un s'inscrit. Le compte n'est pas utilisable tout de suite : il attend la validation de la RH.

Fonctionnement.

1. Vérifie que l'e-mail n'est pas déjà utilisé et que le rôle demandé est employe ou chef (jamais rh ni dg).
2. Hache le mot de passe avec `password_hash`.
3. Enregistre le compte avec le statut `en_attente`.
4. Retourne l'utilisateur créé.

`+ authentifier(email : String, motDePasse : String) : Utilisateur` — ***statique***

Description. Connecte une personne à partir de son e-mail et de son mot de passe. Elle est statique parce qu'on n'a pas encore d'objet Utilisateur au moment de se connecter.

Fonctionnement.

1. Cherche le compte qui a cet e-mail.
2. Vérifie le mot de passe avec `password_verify`.
3. Refuse si le compte n'est pas `actif` (en attente ou désactivé).
4. Si tout est bon : met à jour `derniereConnexion` et retourne l'utilisateur.
5. Sinon retourne `null`, avec le même message d'erreur que l'e-mail soit inconnu ou le mot de passe faux (on ne révèle pas quels e-mails existent).

`+ changerMotDePasse(ancien : String, nouveau : String) : void`

Description. Permet à l'utilisateur connecté de changer son propre mot de passe. Le compte concerné vient de la session, jamais d'un champ saisi : personne ne peut ainsi viser le compte d'un autre.

Fonctionnement.

1. Vérifie que `ancien` correspond au mot de passe actuel.
2. Hache `nouveau` et remplace l'ancien hachage.
3. Remet `doitChangerMdp` à faux (utile après un premier mot de passe imposé).

`+ demanderReinitialisation(email : String) : void` — ***statique***

Description. Lance la procédure « mot de passe oublié ». Statique : la personne n'est pas connectée, elle donne seulement son e-mail.

Fonctionnement.

1. Cherche un compte actif avec cet e-mail.
2. S'il existe : crée un jeton aléatoire à durée limitée, stocke seulement son empreinte, et envoie par e-mail un lien contenant le jeton.
3. Dans tous les cas, l'écran répond la même chose (« si ce compte existe, un e-mail a été envoyé »).

`+ valider(valideur : Utilisateur) : void`

Description. Active un compte en attente. Seule une personne autorisée peut le faire.

Fonctionnement.

1. Vérifie que `valideur` a le droit : la RH valide les employés et les chefs, le DG valide les comptes RH.
2. Refuse si le valideur est la personne elle-même.
3. Passe le statut de `en_attente` à `actif` et enregistre le valideur et la date.

`+ desactiver(auteur : Utilisateur) : void`

Description. Bloque un compte sans jamais le supprimer, pour garder l'historique (congés, paie, journal).

Fonctionnement.

1. Vérifie le droit de `auteur` : la RH désactive employés et chefs, le DG désactive la RH.
2. Passe le statut à `desactive`.
3. Journalise l'action.

`+ peut(permission : String) : boolean`

Description. Dit si l'utilisateur a le droit d'effectuer une action. Appelée avant toute action sensible, pour ne pas dupliquer les contrôles de droits dans chaque page.

Fonctionnement.

1. Prend le rôle de l'utilisateur (employe, chef, rh, dg).
2. Compare avec le tableau des permissions (par exemple : `valider_paie` réservé à rh).
3. Retourne vrai ou faux.

JetonReinitialisation

Le lien à usage unique envoyé par e-mail quand quelqu'un a oublié son mot de passe.

`+ trouverParJeton(jeton : String) : JetonReinitialisation` — ***statique***

Description. Retrouve l'enregistrement correspondant au lien reçu par e-mail. Statique : on part du jeton, pas d'un objet.

Fonctionnement.

1. Calcule l'empreinte (hachage) du jeton reçu.
2. Cherche cette empreinte dans la table.
3. Retourne le jeton trouvé, ou `null` s'il n'existe pas.

`+ estValide() : boolean`

Description. Dit si le lien est encore utilisable.

Fonctionnement.

1. Compare `expireLe` avec l'heure actuelle.

2. Vérifie que `utiliseLe` est vide.
3. Retourne vrai si le lien n'est ni expiré ni déjà utilisé.

`+ consommer(nouveauMotDePasse : String) : void`

Description. Termine la réinitialisation : change le mot de passe et rend le lien inutilisable.

Fonctionnement.

1. Vérifie `estValide()` ; sinon refuse.
2. Change le mot de passe de l'utilisateur lié.
3. Renseigne `utiliseLe` : le lien ne marchera plus une deuxième fois.

Employe

Le dossier du salarié : identité, département, poste, statut. C'est la classe centrale du projet.

`+ getContratEnCours() : Contrat`

Description. Donne le contrat actuellement valable de l'employé. C'est la base du calcul de sa paie.

Fonctionnement.

1. Parcourt les contrats de l'employé.
2. Garde celui au statut `en_cours` dont les dates couvrent aujourd'hui.
3. Retourne ce contrat, ou `null` s'il n'y en a pas.

`+ getSoldeDisponible(type : TypeConge, annee : int) : decimal`

Description. Donne le nombre de jours de congé que l'employé peut encore poser pour un type et une année. Sert à l'écran de demande de congé.

Fonctionnement.

1. Retrouve le `SoldeConge` de l'employé pour ce type et cette année.
2. Appelle `getDisponible()` sur ce solde.
3. Retourne le nombre de jours.

`+ estChef() : boolean`

Description. Dit si l'employé dirige un département. Règle du projet : le rôle `chef` est celui des employés qui dirigent un département.

Fonctionnement.

1. Regarde si l'employé est désigné comme chef dans un département (association « dirige »).
2. Retourne vrai ou faux.

`+ affecter(departement : Departement, poste : Poste, auteur : Utilisateur) : void`

Description. Change le département et le poste de l'employé (mutation, promotion).

Fonctionnement.

1. Garde l'ancienne affectation.
2. Enregistre le nouveau département et le nouveau poste.
3. Journalise l'ancienne et la nouvelle valeur avec `auteur`.

`+ changerStatut(statut : StatutEmploye, auteur : Utilisateur) : void`

Description. Change la situation de l'employé : actif, suspendu ou sorti.

Fonctionnement.

1. Enregistre le nouveau statut.
2. Si le statut devient `sorti` : clôture le contrat en cours et désactive le compte.
3. Journalise le changement avec `auteur`.

`+ rechercher(departement : Departement, poste : Poste, statut : StatutEmploye) : List<Employe>` — ***statique***

Description. Cherche des employés selon des filtres. Statique : elle renvoie une liste, pas un seul employé. Alimente la recherche dynamique (AJAX).

Fonctionnement.

1. Construit la requête SQL avec seulement les filtres renseignés (un filtre à `null` est ignoré).
2. Exécute la requête.
3. Retourne la liste des employés trouvés.

Contrat

Le contrat de travail d'un employé : type, dates, salaire de base.

`+ estEnCours(date : Date) : boolean`

Description. Dit si le contrat est valable à une date donnée.

Fonctionnement.

1. Vérifie que le statut est `en_cours`.
2. Vérifie que la date est après `dateDebut` et avant `dateFin` (si `dateFin` est vide, le contrat est à durée indéterminée).
3. Retourne vrai ou faux.

`+ chevauche(autre : Contrat) : boolean`

Description. Aide à respecter la règle : un employé n'a qu'un seul contrat en cours à la fois.

Fonctionnement.

1. Compare les périodes des deux contrats.
2. Retourne vrai si elles se recouvrent.

`+ getTauxHoraire() : decimal`

Description. Donne le prix d'une heure de travail, base du calcul des heures supplémentaires.

Fonctionnement.

1. Divise `salaireBase` par `heuresMensuelles`.
2. Retourne le résultat.

`+ clore(dateFin : Date, resilie : boolean) : void`

Description. Termine le contrat, soit à l'échéance, soit par résiliation.

Fonctionnement.

1. Enregistre `dateFin`.
2. Passe le statut à `termine` (échéance) ou `resilie` (si `resilie` est vrai).
3. Plus aucune fiche de paie ne peut être créée après `dateFin`.

`+ exporterWord() : File`

Description. Produit le contrat sous forme de fichier Word.

Fonctionnement.

1. Remplit un modèle Word avec les informations du contrat et de l'employé (bibliothèque PhpWord).
2. Enregistre le fichier et garde son chemin dans `fichierChemin`.
3. Retourne le fichier.

Departement

Un service de l'entreprise, avec un chef et éventuellement des sous-départements.

`+ getEffectif() : int`

Description. Compte les employés actifs du département (tableau de bord).

Fonctionnement.

1. Compte les employés du département dont le statut est `actif`.
2. Retourne le nombre.

`+ getMasseSalariale(annee : int, mois : int) : decimal`

Description. Calcule le total des salaires bruts d'un département pour un mois.

Fonctionnement.

1. Prend les fiches de paie validées du mois, pour les employés du département.
2. Additionne leur `salaireBrut`.
3. Retourne la somme.

`+ designerChef(employe : Employe, auteur : Utilisateur) : void`

Description. Nomme le chef du département.

Fonctionnement.

1. Retire le rôle `chef` à l'ancien chef (il redevient `employe`).
2. Enregistre le nouveau chef et lui donne le rôle `chef`.
3. Journalise le changement avec `auteur`.

`+ getSousDepartements() : List<Departement>`

Description. Donne les départements placés directement sous celui-ci. Sert à l'organigramme dépliable.

Fonctionnement.

1. Cherche les départements dont le parent est ce département.
2. Retourne la liste.

Poste

Une fonction (comptable, développeur…) à laquelle des primes peuvent être attachées.

`+ getPrimes() : List<PosteRubrique>`

Description. Donne les primes prévues pour ce poste. Elles sont ajoutées automatiquement à chaque fiche de paie.

Fonctionnement.

1. Cherche les lignes `PosteRubrique` liées au poste.
2. Retourne la liste.

DemandeModification

Une demande de correction d'une information du dossier, que la RH accepte ou refuse.

`+ accepter(rh : Utilisateur) : void`

Description. La RH accepte la correction demandée par l'employé sur son dossier.

Fonctionnement.

1. Applique `nouvelleValeur` au champ concerné du dossier.
2. Passe le statut à `acceptee` et enregistre la date de traitement et la RH.
3. Journalise l'ancienne et la nouvelle valeur.
4. Notifie l'employé.

`+ refuser(rh : Utilisateur, motif : String) : void`

Description. La RH refuse la correction.

Fonctionnement.

1. Passe le statut à `refusee`.
2. Le dossier n'est pas modifié.
3. Notifie l'employé en lui indiquant le motif.

DocumentEmploye

Un fichier privé rangé dans le dossier de l'employé (pièce d'identité, diplôme, attestation…).

`+ estAccessiblePar(utilisateur : Utilisateur) : boolean`

Description. Dit si une personne a le droit de voir ce document. Les fichiers sont privés.

Fonctionnement.

1. Autorise le propriétaire du document, la RH et le DG.
2. Refuse tous les autres.
3. Retourne vrai ou faux.

`+ telecharger(utilisateur : Utilisateur) : File`

Description. Envoie le fichier à une personne autorisée.

Fonctionnement.

1. Appelle `estAccessiblePar(utilisateur)` ; refuse si faux.
2. Lit le fichier dans le dossier privé (`cheminPrive`) et l'envoie via un script PHP (le fichier n'est jamais accessible par une adresse directe).
3. Journalise le téléchargement.

DemandeDocument

Une demande de document (par exemple une attestation) adressée à la RH.

`+ generer(rh : Utilisateur) : DocumentEmploye`

Description. La RH répond à une demande d'attestation en créant le document.

Fonctionnement.

1. Produit le fichier Word de l'attestation à partir d'un modèle.
2. L'ajoute au coffre-fort de l'employé (nouveau `DocumentEmploye`).
3. Passe le statut à `generee`.
4. Notifie l'employé.

`+ refuser(rh : Utilisateur, motif : String) : void`

Description. La RH refuse la demande de document.

Fonctionnement.

1. Passe le statut à `refusee`.
2. Notifie l'employé en lui indiquant le motif.

Absence

Une absence constatée par le chef pour un jour donné, éventuellement rattachée à un congé.

`+ marquer(employe : Employe, date : Date, motif : String, chef : Utilisateur) : Absence` — ***statique***

Description. Le chef note qu'un membre de son équipe est absent. Statique : l'absence n'existe pas encore.

Fonctionnement.

1. Vérifie que `chef` dirige le département de l'employé.
2. Refuse si une absence existe déjà pour cet employé à cette date.
3. Crée l'absence avec le motif et la date.

`+ justifier(motif : String) : void`

Description. Indique que l'absence est justifiée.

Fonctionnement.

1. Passe `justifiee` à vrai.
2. Garde le motif.

`+ rattacherAConge(demande : DemandeConge) : void`

Description. Relie l'absence à un congé approuvé qui la couvre.

Fonctionnement.

1. Vérifie que le congé est `approuvee` et couvre la date.
2. Enregistre le lien et passe l'absence à justifiée.

JourFerie

Un jour férié, exclu du décompte des jours de congé.

`+ estOuvrable(date : Date) : boolean` — ***statique***

Description. Dit si un jour compte comme jour de travail. Statique : on la demande pour une date, sans objet.

Fonctionnement.

1. Retourne faux pour le samedi et le dimanche.

2. Retourne faux si la date est un jour férié (les fériés qui reviennent chaque année sont comparés par jour et mois).
3. Sinon retourne vrai.

`+ getFeriesEntre(debut : Date, fin : Date) : List<Date>` — ***statique***

Description. Donne les jours fériés entre deux dates. Le navigateur s'en sert pour afficher la durée en direct (AJAX) pendant que l'employé choisit ses dates.

Fonctionnement.

1. Cherche les jours fériés entre `debut` et `fin`, y compris les récurrents.
2. Retourne la liste des dates.

DemandeConge

Une demande de congé et son circuit de validation.

`+ calculerNbJours() : decimal`

Description. Compte les jours réellement décomptés pour la demande.

Fonctionnement.

1. Parcourt les dates de `dateDebut` à `dateFin`.
2. Ne garde que les jours ouvrables (sans week-ends ni jours fériés, via `JourFerie.estOuvrable`).
3. Compte une demi-journée comme 0,5.
4. Enregistre le résultat dans `nbJours` et le retourne.

`+ verifierSolde() : boolean`

Description. Vérifie que l'employé a assez de jours.

Fonctionnement.

1. Si le type de congé ne décompte pas le solde, retourne vrai.
2. Sinon compare `nbJours` à `SoldeConge.getDisponible()`.
3. Retourne vrai si `nbJours` est inférieur ou égal.

`+ aChevauchement() : boolean`

Description. Empêche de poser deux congés sur les mêmes jours.

Fonctionnement.

1. Cherche les autres demandes de l'employé, ni refusées ni annulées.
2. Retourne vrai si l'une couvre une partie de la période.

`+ soumettre() : void`

Description. Envoie la demande pour décision. C'est le point de départ du circuit de validation.

Fonctionnement.

1. Calcule `nbJours`.
2. Vérifie la durée, le solde, l'absence de chevauchement et la présence du justificatif si le type l'exige ; refuse sinon.
3. Fixe le statut de départ avec `determinerStatutInitial()`.
4. Notifie le premier décideur et journalise.

`- determinerStatutInitial() : StatutConge` — ***privée***

Description. Choisit à qui la demande va d'abord, selon le rôle du demandeur. Méthode interne de `soumettre`.

Fonctionnement.

1. Employé : statut `en_attente_chef`.
2. Chef : statut `en_attente_rh`.
3. Membre de la RH : statut `en_attente_dg`.

`+ donnerAvis(chef : Utilisateur, favorable : boolean, commentaire : String) : ValidationConge`

Description. Le chef donne son avis sur la demande d'un employé de son équipe (première étape).

Fonctionnement.

1. Vérifie que le demandeur est un employé du département du chef.
2. Crée une `ValidationConge` à l'étape `chef`, avec `favorable` ou `defavorable`.
3. Passe la demande à `en_attente_rh`.
4. Notifie la RH.

`+ decider(decideur : Utilisateur, approuve : boolean, commentaire : String) : ValidationConge`

Description. Décision finale : la RH (ou le DG pour une demande venant de la RH) approuve ou refuse. Tout est fait en une seule transaction : tout réussit ou rien n'est enregistré.

Fonctionnement.

1. Vérifie que le décideur est autorisé et n'est pas le demandeur.
2. Crée la `ValidationConge` (étape `rh` ou `dg`).
3. Passe le statut à `approuvee` ou `refusee`.
4. Si approuvée : appelle `SoldeConge.debiter(nbJours)`.

5. Notifie l'employé (notification et e-mail) et journalise.

`+ annuler(auteur : Utilisateur) : void`

Description. Annule la demande.

Fonctionnement.

1. Vérifie le droit de `auteur` : le demandeur, ou la RH.
2. Passe le statut à `annulee`.
3. Si la demande était `approuvee` : appelle `SoldeConge.recrediter(nbJours)`.
4. Journalise l'action.

`+ estEnCoursLe(date : Date) : boolean`

Description. Dit si l'employé est en congé à une date donnée. Alimente l'écran « Congés en cours ».

Fonctionnement.

1. Vérifie que le statut est `approuvee`.
2. Vérifie que la date est entre `dateDebut` et `dateFin`.

`+ estEnAttente() : boolean`

Description. Dit si la demande n'est pas encore tranchée.

Fonctionnement.

1. Retourne vrai si le statut est `en_attente_chef`, `en_attente_rh` ou `en_attente_dg`.

ValidationConge

Une étape du circuit : l'avis du chef, la décision de la RH ou du DG.

`+ estCoherente() : boolean`

Description. Vérifie que la décision correspond à l'étape.

Fonctionnement.

1. Étape `chef` : la décision doit être `favorable` ou `defavorable` (un avis).
2. Étapes `rh` et `dg` : la décision doit être `approuve` ou `refuse`.

`+ estAutorisee() : boolean`

Description. Vérifie que la personne qui valide en a le droit.

Fonctionnement.

1. Le décideur n'est pas le demandeur.
2. Étape `chef` : l'avis vient du chef du département de l'employé.
3. Étapes `rh` et `dg` : la décision vient d'un compte du rôle correspondant.

SoldeConge

Les jours de congé d'un employé pour un type et une année : acquis, pris, disponibles.

`- getJoursEnAttente() : decimal` — ***privée***

Description. Compte les jours déjà réservés par des demandes pas encore tranchées. Méthode interne de `getDisponible`.

Fonctionnement.

1. Prend les demandes de l'employé, du même type et de la même année, dont le statut est `en_attente_chef`, `en_attente_rh` ou `en_attente_dg`.
2. Additionne leur `nbJours`.

`+ getDisponible() : decimal`

Description. Donne les jours que l'employé peut encore poser. Le résultat est calculé à chaque appel, jamais stocké, donc toujours à jour.

Fonctionnement.

1. Calcule `joursAcquis - joursPris - getJoursEnAttente()`.
2. Retourne le résultat.

`+ debiter(nbJours : decimal) : void`

Description. Retire des jours du solde quand une demande est approuvée.

Fonctionnement.

1. Refuse si le solde est insuffisant (pour les types décomptés).
2. Ajoute `nbJours` à `joursPris`.

`+ recrediter(nbJours : decimal) : void`

Description. Rend des jours au solde quand un congé approuvé est annulé.

Fonctionnement.

1. Retire `nbJours` de `joursPris`.

`+ crediter(nbJours : decimal) : void`

Description. Ajoute des jours acquis (par exemple 1,5 jour par mois, selon le droit annuel du type).

Fonctionnement.

1. Ajoute `nbJours` à `joursAcquis`.

TypeConge

Une sorte de congé (annuel, maladie…) et ses règles : justificatif, décompte du solde, droit annuel.

`+ desactiver() : void`

Description. Retire un type de congé de la liste des choix sans le supprimer (les anciennes demandes gardent leur type).

Fonctionnement.

1. Passe `actif` à faux.

FichePaie

Le bulletin d'un employé pour un mois, de brouillon jusqu'à l'archivage.

`+ estModifiable() : boolean`

Description. Dit si la fiche peut encore être changée.

Fonctionnement.

1. Retourne vrai si le statut est `brouillon` ou `verifiee`.
2. Après validation, la fiche est en lecture seule.

`+ calculer() : void`

Description. Construit la fiche du mois : lignes puis totaux. Seulement en brouillon.

Fonctionnement.

1. Ajoute les primes du poste et les primes personnelles actives du mois.
2. Ajoute les cotisations (taux pris dans `ParametreSysteme`).
3. Intègre les heures supplémentaires validées et les avances à déduire.
4. Calcule `salaireBrut`, `totalPrimes`, `totalRetenues`, puis le net avec `calculerNet()`.

`+ ajouterLigne(ligne : LignePaie) : void`

Description. Ajoute à la main une prime ou une retenue exceptionnelle.

Fonctionnement.

1. Refuse si `estModifiable()` est faux.
2. Exige une justification.
3. Ajoute la ligne et recalcule les totaux.

`+ calculerNet() : decimal`

Description. Calcule le net à payer.

Fonctionnement.

1. Calcule `salaireBrut + totalPrimes - totalRetenues` (formule imposée par le cahier des charges).
2. Retourne le résultat.

`+ detecterAnomalies() : List<String>`

Description. Repère les fiches suspectes avant validation.

Fonctionnement.

1. Compare le net au mois précédent : signale un écart supérieur au seuil réglé dans `ParametreSysteme`.
2. Signale un net négatif ou nul.
3. Signale une ligne exceptionnelle sans justification.
4. Retourne la liste des messages d'alerte.

`+ verifier() : void`

Description. Étape de contrôle.

Fonctionnement.

1. Contrôle les anomalies détectées.
2. Passe le statut de `brouillon` à `verifiee`.

`+ valider(rh : Utilisateur) : void`

Description. La RH valide la fiche, qui est alors figée.

Fonctionnement.

1. Vérifie que le statut est `verifiee`.
2. Passe à `validee` et enregistre le validateur et la date.
3. Journalise l'action.

`+ archiver() : void`

Description. Range la fiche une fois la période terminée.

Fonctionnement.

1. Passe le statut de `validee` à `archivee`.

`+ genererPDF() : File`

Description. Produit le bulletin de paie en PDF.

Fonctionnement.

1. Vérifie que la fiche est validée.
2. Génère le PDF avec Dompdf.
3. Enregistre le fichier et garde son chemin dans `pdfChemin`.

`+ envoyerParEmail() : boolean`

Description. Envoie le bulletin à l'employé.

Fonctionnement.

1. Génère le PDF s'il n'existe pas.
2. Envoie l'e-mail avec PHPMailer et joint le PDF.
3. Renseigne `envoyeeLe` et crée une notification.

LignePaie

Une ligne du bulletin : une prime ou une retenue avec son montant.

`+ calculerMontant() : decimal`

Description. Calcule le montant d'une ligne de la fiche.

Fonctionnement.

1. Si le mode est `fixe` : le montant est la valeur donnée.
2. Si le mode est `pourcentage` : `baseCalcul × taux ÷ 100`.
3. Le résultat est enregistré tel quel : c'est un instantané, qui ne change pas si le taux change plus tard.

RubriquePaie

Un modèle de prime ou de retenue (prime d'ancienneté, cotisation…).

`+ calculerMontant(base : decimal) : decimal`

Description. Calcule le montant d'une rubrique à partir d'une base.

Fonctionnement.

1. Mode `fixe` : retourne `valeurDefaut`.
2. Mode `pourcentage` : retourne `base × valeurDefaut ÷ 100`.

`+ desactiver() : void`

Description. Retire la rubrique des choix sans la supprimer (les anciennes fiches gardent leurs lignes).

Fonctionnement.

1. Passe `actif` à faux.

HeureSupplementaire

Des heures supplémentaires saisies par l'employé, validées par le chef, payées via la fiche.

`+ calculerMontant() : decimal`

Description. Calcule ce que valent les heures supplémentaires.

Fonctionnement.

1. Prend le taux horaire du contrat en cours de l'employé.
2. Calcule `nbHeures × tauxHoraire × tauxMajoration`.
3. Retourne le montant.

`+ valider(chef : Utilisateur) : void`

Description. Le chef valide les heures supplémentaires de son équipe.

Fonctionnement.

1. Vérifie que `chef` dirige le département de l'employé.
2. Copie le taux de majoration actuel (il ne changera plus pour cette saisie) et calcule le montant.
3. Passe le statut de `saisie` à `validee_chef`.

`+ rejeter(chef : Utilisateur, motif : String) : void`

Description. Le chef refuse les heures.

Fonctionnement.

1. Passe le statut à `rejetee`.
2. Notifie l'employé en lui indiquant le motif.

`+ integrerA(fiche : FichePaie) : void`

Description. Ajoute les heures validées à une fiche de paie.

Fonctionnement.

1. Vérifie que le statut est `validee_chef`.
2. Relie les heures à la fiche et passe le statut à `integree`.

AvanceSalaire

Une avance sur salaire, accordée par la RH puis déduite d'une fiche de paie.

`+ accorder(rh : Utilisateur) : void`

Description. La RH accorde l'avance demandée.

Fonctionnement.

1. Passe le statut de `en_attente` à `accordee`.
2. Notifie l'employé.

`+ refuser(rh : Utilisateur, motif : String) : void`

Description. La RH refuse l'avance.

Fonctionnement.

1. Passe le statut à `refusee`.
2. Notifie l'employé en lui indiquant le motif.

`+ deduireSur(fiche : FichePaie) : LignePaie`

Description. Retire l'avance du salaire sur une fiche.

Fonctionnement.

1. Crée une retenue d'origine `avance` sur la fiche du mois.
2. Relie la fiche à l'avance.
3. Retourne la ligne.

EmployeRubrique

Une prime personnelle accordée à un employé, avec ses dates.

`+ estActive(date : Date) : boolean`

Description. Dit si la prime personnelle s'applique à une date.

Fonctionnement.

1. Vérifie `dateDebut ≤ date ≤ dateFin` (si `dateFin` est vide, sans limite).
2. Retourne vrai ou faux.

`+ genererLigne(fiche : FichePaie) : LignePaie`

Description. Crée la ligne de prime personnelle sur une fiche.

Fonctionnement.

1. Vérifie `estActive` pour le mois de la fiche ; sinon ne fait rien.
2. Crée une `LignePaie` d'origine `employe`.
3. Retourne la ligne.

PosteRubrique

Une prime prévue pour tous les occupants d'un poste.

`+ genererLigne(fiche : FichePaie) : LignePaie`

Description. Crée la ligne de prime prévue pour un poste sur une fiche.

Fonctionnement.

1. Calcule le montant prévu pour ce poste.
2. Crée une `LignePaie` d'origine `poste` sur la fiche.
3. Retourne la ligne.

Notification

Un message interne (et éventuellement un e-mail) envoyé à un utilisateur.

`+ notifier(destinataire : Utilisateur, type : String, titre : String, message : String, lien : String) : Notification` — ***statique***

Description. Prévient quelqu'un d'un événement. Statique : on crée la notification sans objet de départ.

Fonctionnement.

1. Crée la notification interne (titre, message, lien).
2. Si l'événement prévoit un e-mail : l'envoie avec PHPMailer et renseigne `emailEnvoye`.
3. Retourne la notification.

`+ marquerLue() : void`

Description. Marque la notification comme lue.

Fonctionnement.

1. Passe `lue` à vrai quand l'utilisateur ouvre le lien.

JournalAction

L'historique des actions sensibles : qui a fait quoi, quand, avec quelles valeurs. On y ajoute des lignes, on n'en modifie jamais.

`+ enregistrer(auteur : Utilisateur, action : String, entite : String, entiteId : int, ancienne : String, nouvelle : String) : void` — ***statique***

Description. Écrit une ligne dans l'historique des actions. Statique et la seule écriture possible : on n'a ni modification ni suppression, même pour le DG.

Fonctionnement.

1. Reçoit l'auteur, l'action, l'entité concernée et les valeurs avant/après.
2. Ajoute une ligne avec la date et l'heure actuelles.

`+ rechercher(auteur : Utilisateur, entite : String, du : Date, au : Date) : List<JournalAction>` — ***statique***

Description. Consulte l'historique.

Fonctionnement.

1. Applique les filtres (entité, dates).
2. La RH ne voit que les actions sur les employés ; le DG voit tout.
3. Retourne la liste.

ParametreSysteme

Les réglages modifiables : taux de cotisation, majoration des heures supplémentaires, seuil d'anomalie, droit annuel par défaut.

`+ getDecimal(cle : String) : decimal` — ***statique***

Description. Lit un réglage numérique. Statique : on lit un réglage par son nom (taux de cotisation, majoration des heures supplémentaires, seuil d'anomalie, droit annuel…).

Fonctionnement.

1. Cherche le réglage dont la clé est `cle`.
2. Convertit la valeur en nombre et la retourne.

`+ modifier(valeur : String, auteur : Utilisateur) : void`

Description. Change la valeur d'un réglage.

Fonctionnement.

1. Remplace `valeur` et met à jour `dateModification`.
2. Journalise l'ancienne et la nouvelle valeur avec `auteur`.

6. Contraintes d'unicité (reprises du MCD)

- SoldeConge : une seule ligne par (employé, type de congé, année)
- FichePaie : une seule fiche par (employé, année, mois)
- Absence : une seule absence par (employé, jour)
- ValidationConge : une seule validation par (demande, étape)